

C++&Qt Day03 hw2 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.07

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에서 A*알고리즘과 Dijkstra 알고리즘을 이용한 최단 경로 탐색 프로그램을 Qt로 구현하는 과정에서 chatGPT를 사용하였다.

아직 Qt 프로그램을 학습하고 있기 때문에 프로그램 코드를 처음부터 직접 작성하는 데 어려움이 있었다. 특히 A*와 Dijkstra 알고리즘의 구현, priority_queue 사용, 탐색 결과 저장, QTimer를 사용한 탐색 과정 시각화 부분 코드 작성할때 AI를 사용하였다.

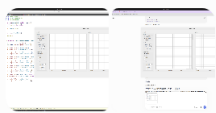
2. AI 사용 부분

2.1 맵 생성 및 편집 기능

격자 맵을 구현하기 위해 QTableWidgetItem를 사용하는 방법을 AI에게 질문하였다.

Rows와 Cols 값을 이용하여 테이블의 행과 열을 변경하고, 한쪽에서 장애물, 시작점, 도착점을 조작하더라도 양쪽 A*와 Dijkstra 화면에 동일한 맵이형성되도록 구현하는데 도움을 받았다.

또한 QHeaderView::Stretch를 사용하여 맵 크기에 따라 셀 크기가 자동으로 조절 되도록 하는데도 AI의 도움을 받았다.



사소한 문제가 있어 2020은 잘 되는데 27 27로 시험해보니까 팔려서 스크롤 해야 전체를 볼 수있네

ㅇㅇ, 원인 보여. 27*27에서 셀이 일정 크기 이하로 줄어들지 못해서 QTableWidgetItem 이 스크롤바를 만들어버린 거야.

우리는 참고 영상처럼 맵 크기에 맞춰 셀들이 자동으로 작아지게 하는 게 더 좋으니까 바로 고치자.

on_createMapButton_clicked()에서 QHeaderView::Stretch 설정하기 전에 아래를 추가해.

```
C++
ui->astarTable->horizontalHeader()->setMinimumSectionSize(1);
ui->astarTable->verticalHeader()->setMinimumSectionSize(1);

ui->dijkstraTable->horizontalHeader()->setMinimumSectionSize(1);
ui->dijkstraTable->verticalHeader()->setMinimumSectionSize(1);
```

그리고 스크롤바도 아예 없애자.

```
C++
ui->astarTable->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
ui->astarTable->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);

ui->dijkstraTable->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
ui->dijkstraTable->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
```

그래서 해당 부분은 이렇게 되는 거야.

```
C++
ui->astarTable->horizontalHeader()->setMinimumSectionSize(1);
ui->astarTable->verticalHeader()->setMinimumSectionSize(1);

ui->dijkstraTable->horizontalHeader()->setMinimumSectionSize(1);
ui->dijkstraTable->verticalHeader()->setMinimumSectionSize(1);

ui->astarTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);
ui->astarTable->verticalHeader()->setSectionResizeMode(QHeaderView::Stretch);

ui->dijkstraTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);
ui->dijkstraTable->verticalHeader()->setSectionResizeMode(QHeaderView::Stretch);

ui->astarTable->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
ui->astarTable->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);

ui->dijkstraTable->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
ui->dijkstraTable->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
```

왜 이러나면

원래:

ㅇㅇ. 이제 *첫 코드* Create Map 버튼 누르면 A / Dijkstra 양쪽에 같은 크기의 격자 생성**부터 하자.

지금은 알고리즘 전혀 안 넣고, 20*20 맵이 화면에 뜨는 것만 만들 거야.

1. mainwindow.h

왼쪽에서 mainwindow.h 열고 private slots:를 추가해.

기본 코드가 대충 이렇게 되어 있을 텐데:

```
C++
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

QT_BEGIN_NAMESPACE
namespace UI {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_createMapButton_clicked();

private:
    UI::MainWindow *ui;
};

#endif
```

핵심은 이것 하나야.

```
C++
void on_createMapButton_clicked();
```

createMapButton을 클릭했을 때 실행할 함수라는 뜻이야.

2. mainwindow.cpp

이제 mainwindow.cpp 열어.

위쪽에:

```
C++
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QHeaderView>
```

2.2 A* 및 Dijkstra 알고리즘 구현

A* 및 Dijkstra 알고리즘을 구현할때 AI의 도움을 받았다. hw1 보고서를 작성하며 두개의 알고리즘의 개념은 알고있었지만, 실제 C++코드로 구현하려 하니 익숙치 않아어려움이 있었다. 따라서 AI의 도움을 받아 Dijkstra 알고리즘은 각 노드까지의 최소 비용을 저장하고 priority_queue를 이용하여 현재까지의 비용이 가장 적은 노드를 먼저 탐색하는 구조로 구현하였고, A*알고리즘 또한 AI의 도움을 받아 $f(n) = g(n) + h(n)$ 을 이용하여 코드를 작성하였다.

좋아. 이제 Dijkstra부터 하나씩 붙이자.

이번 단계 목표는:

시작점/도착점/장애물을 만든 뒤 Apply를 누르면 Dijkstra 쪽에서 탐색한 칸 + 최단 경로 + Cost / Visited / Time이 표시되게 만들기.

과제에서 요구하는 비교 항목도 최종 경로 비용 / 탐색한 노드 수 / 실행 시간 이야. CQC_day3

헛갈리지 않게 이번에도 mainwindow.h, mainwindow.cpp 전체 코드로 줄게.

1. mainwindow.h 전체

```
// C++
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QTableWidget>

#include <vector>
#include <utility>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_createMapButton_clicked();

    void on_astarTable_cellClicked(int row, int column);
    void on_dijkstraTable_cellClicked(int row, int column);

    void on_applyButton_clicked();

private:
    Ui::MainWindow *ui;

    // 맵 데이터
    // 0 : 빈칸
    // 1 : 장애물
    // 2 : 시작점
    // 3 : 도착점
    std::vector<std::vector<int>> mapData;
```

좋아. 이제 왼쪽 A*까지 붙여서 A*와 Dijkstra가 같은 맵에서 동시에 탐색되는 형태로 만들자. PPT에서도 A*는 $f(n)=g(n)+h(n)$ 이고, Dijkstra는 $f(n)=g(n)$ 으로 비교하고 있어. CQC_day3

이번에 바뀌는 핵심은 3개야.

- runAStar() 추가
- Apply 누르면 A*와 Dijkstra 둘 다 계산
- 하나의 QTimer로 양쪽 탐색을 동시에 한 칸씩 시각화

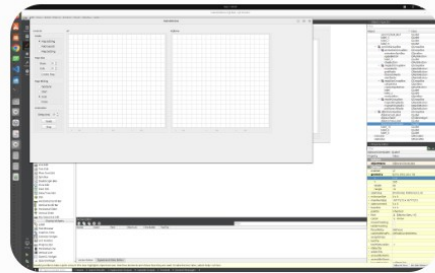
2.3 오류 수정 및 테스트

첫번째로 Cost: 라고 표시되어야 할 글자에 QLabel 에 결과 출력용 objectName 을 잘못 설정하여 Cost:라는 글자 자체가 - 로 변경되는 문제가 있었다.

두번째는 초기 A* 구현에서 탐색 영역이 Dijkstra 와 차이하지 않는 문제였다.

이러한 문제들은 프로그램을 실행 시켜가며 발견하였고 AI 에게 질문한 뒤 원인을 이해하고 수정하였다.

이 외에도 전반적인 코드 수정 및 작성에서 AI 의 도움을 받았다.



이거 수정해줘 원래 cost: 옆에 숫자가떠야하는데 cost 자체가 - 이걸로 바뀌었어

맞아. 이걸 알고리즘 문제가 아니라 UI에서 QLabel 의 objectName 을 반대로 붙인 문제야.

지금 코드에서:

```
</> C++  
ui->astarCostLabel->setText("-");  
ui->astarVisitedLabel->setText("-");  
ui->astarTimeLabel->setText("-");
```

이렇게 하는데, 네 Designer에서 astarCostLabel 이 Cost: 글자가 적힌 QLabel에 붙어 있어서 Cost: 자체가 - 로 바뀐 거야.

원래 원하는 건:

```
Cost: -  
Visited: -  
Time: -
```

이거지.

Designer에서 이렇게 고쳐

A* 아래 QLabel 6개를 구분해.

글자 설명용 QLabel 3개

```
Cost:  
Visited:  
Time:
```

얘네 objectName은 아무거나 해도 돼. 예:

```
astarCostTextLabel  
astarVisitedTextLabel  
astarTimeTextLabel
```

그리고 오른쪽 숫자 출력용 QLabel 3개:



3. 깨달은 점

A* 와 Dijkstra 의 차이를 단순히 글로 읽고 이해했을때 보다 직접적으로 탐색 과정을 화면으로 확인하면서 비교하니 더 쉽게 이해할 수 있었다.

Dijkstra 는 현재까지의 실제 비용만을 기준으로 하기 때문에 시작점 주변을 넓게 탐색하지만, A*는 목적지까지의 예상 비용을 함께 사용하여 목적지 방향을 중심으로 탐색하는 차이점을 알 수 있었다.

또한 이번 과제에서는 코드 작성에 AI 의 도움을 많이 받은건 사실이지만, 직접 프로그램을 실행하고 문제를 찾고 수정 과정을 반복하면서 단순히 AI 가 작성한 결과를 사용하는 것보다 코드가 어떤 역할을 하는지 확인하는 과정이 중요하다는 점을 느꼈다.